

MACHINE LEARNING PREPROCESSING MASTER REFERENCE

Master Reference Guide:

ml_preprocessing_master_reference.py

Author: Ankur Mittal | **Project:** Universal Customer Churn Prediction

1. Pipeline Architecture & Workflow

Full 16-step production pipeline: Data Loading → Inspection → Imputation → Encoding → Scaling → Train-Test Split → Logistic Regression → Probabilities → Custom Thresholding (0.30) → Metrics Evaluation.

2. Data Cleaning & Encoding Nuances

- **Median Imputation:** Monetary features (Total_Charges_INR) use Median to avoid outlier distortion.
- **One-Hot Encoding:** `pd.get_dummies(drop_first=True)` avoids Dummy Variable Trap (Multicollinearity).
- **Automatic Detection:** `df.select_dtypes(include="object")` automatically finds string columns (must `.remove("CustomerID")`).

3. Feature Scaling & Data Leakage Warning

💡 Data Leakage Interview Nuance

Fitting `StandardScaler` on the combined dataset before `train_test_split` leaks future test set statistics into training. In strict production setup, apply `fit_transform()` only on `X_train` and `transform()` on `X_test`.

4. Threshold Tuning & Recall Optimization

Lowering the risk decision threshold from **0.50 to 0.30** increases **Recall to 83.56%**. In Churn prediction, False Negatives (losing a customer worth

₹20,000) are far more expensive than False Positives (sending a ₹100 discount coupon).

5. Complete Master Python Reference Code

```
import sys
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load & Clean
df = pd.read_csv("data/customer_churn_data.csv")
df["Total_Charges_INR"] = df["Total_Charges_INR"].fillna(df["Total_Charges_INR"].median())

# 2. Encode & Scale
cat_cols = ["Gender", "Contract_Type", "Payment_Method", "Tech_Support", "Payment_Problem"]
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True)

num_cols = ["Age", "Tenure_Months", "Monthly_Charges_INR", "Total_Charges_INR"]
scaler = StandardScaler()
df_encoded[num_cols] = scaler.fit_transform(df_encoded[num_cols])

# 3. Split & Train
X = df_encoded.drop(columns=["CustomerID", "Churn"])
y = df_encoded["Churn"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

# 4. Custom Threshold (0.30 Risk Alert)
y_pred_custom = (model.predict_proba(X_test)[:, 1] >= 0.30).astype(int)
print("Recall Score:", recall_score(y_test, y_pred_custom))
```